

Question description

Teddy is visiting the country wonderland. wonderland has n cities and m bi-directional roads. There are k types of tokens. Token i costs c_i . The costs of the tokens are such that for all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$. For each road, you need to have a particular set of tokens, if you want to travel it. Note that you don't have to give the tokens, you just need to show them. Thus, one token can be used at any number of roads, where it is required. Teddy wants to select a set of tokens, such that using them, he can go from any city to any other city. You have to help him minimize the total cost of tokens he buys.

Constraints

$$1 \leq n \leq 10^5$$

$$1 \leq m \leq 10^5$$

No road connects a city to the same city. However, there can be multiple roads between two cities.

$$1 \leq c_i \leq 10^{18}$$

For all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$

Input:

- The first line contains three space separated integers, n m and k .
- The second line contains k space separated integers, where the i^{th} integer denotes the price of i^{th} token, i.e. c_i .
- i^{th} of the next m lines contains three integers u_i, v_i, l_i , where l_i is the number of tokens required by the i^{th} road, followed by l_i indices denoting the tokens required. This road connects cities u_i and v_i

Output:

- Print one integer containing the minimum cost of tokens Teddy has to buy, such that he can travel from any city to any other city. If it is impossible to choose such a set of tokens, print -1 .

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4

EXPECTED OUTPUT

8

Test Case 2

INPUT (STDIN)

3 3 4
11 21 15 10
1 21 2 11 2
1 13 1 3
2 31 1 4

EXPECTED OUTPUT

57

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=1;i<=n;++i)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

480

Test Case 3

NLOC

60

Graph1.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 100005
5
6 typedef struct {
7     int u, v;
8     int l;
9     int *tokens;
10 } Road;
11
12 int parent[MAXN];
13
14 int find(int i) {
15     if (parent[i] == i)
16         return i;
17     return parent[i] = find(parent[i]);
18 }
19
20 int is_connected(int n, Road *roads, int m, int *banned_token) {
21     int i;
22     for(i=0;i<MAXN;++i) {
23         parent[i] = i;
24     }
25     int comps = n;
```

```
26     for (i = 0; i < m; i++) {
27         int valid = 1;
28         for (int j = 0; j < roads[i].l; j++) {
29             if (banned_token[roads[i].tokens[j]]) {
30                 valid = 0;
31                 break;
32             }
33         }
34         if (valid) {
35             int root_u = find(roads[i].u);
36             int root_v = find(roads[i].v);
37             if (root_u != root_v) {
38                 parent[root_u] = root_v;
39                 comps--;
40             }
41         }
42     }
```

```
43     return comps == 1;
44 }
45
```

main | CH.SC.U4CSE25225 | Graph1.c

Ln 90, Col 1 Spaces: 4 UTF-8 CRLF C

```
47     int n, m, k;
48     if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;
49
50     long long *c = (long long*)malloc((k + 1) * sizeof(long long));
51     for (int i = 1; i <= k; i++) {
52         if (scanf("%lld", &c[i]) != 1) {
53             c[i] = 0;
54         }
55     }
56
57     Road *roads = (Road*)malloc(m * sizeof(Road));
58     for (int i = 0; i < m; i++) {
```

```

59     if (scanf("%d %d %d", &roads[i].u, &roads[i].v, &roads[i].l) != 3) {
60         roads[i].u = 0;
61         roads[i].v = 0;
62         roads[i].l = 0;
63     }
64     roads[i].tokens = (int*)malloc(roads[i].l * sizeof(int));
65     for (int j = 0; j < roads[i].l; j++) {
66         if (scanf("%d", &roads[i].tokens[j]) != 1) {
67             roads[i].tokens[j] = 0;
68         }
69     }
70 }
71
72 int *banned = (int*)calloc(MAXN, sizeof(int));
73 if (!is_connected(n, roads, m, banned)) {
74     printf("-1\n");

```

```
75     return 0;
76 }
77
78 long long total_cost = 0;
79 for (int i = k; i >= 1; i--) {
80     banned[i] = 1;
81     if (!is_connected(n, roads, m, banned)) {
82         banned[i] = 0;
83         total_cost += c[i];
84     }
85 }
86
87 printf("%lld\n", total_cost);
88 return 0;
89}
90
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 3 4

1 2 5 10

1 2 2 1 2

1 3 1 3

2 3 1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph1.exe

8

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

3 3 2

4 8

1 2 1 1

2 3 1 2

1 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph1.exe

8

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>